

Life Is Learned - Handoff 005

Library Platform, Discovery, Offload, Dive Deeper, Completion Feedback, and Brand Integration

Baseline branch: feature/handoff-004-6-six-stage-lessons

Baseline SHA: 74a18997dbf8931366ae5acd18f5a4a23ed8699d

Create branch: feature/handoff-005-library-platform

Reviewer: Clyde

Status: Ready for implementation

Merge rule: Do not merge automatically. Push implementation + evidence and stop for reviewer approval.

1. Why Handoff 005 exists

Handoff 004.6 successfully established the six-stage core lesson experience. Real-device testing of *Thinking, Fast and Slow* and *Influence: The Psychology of Persuasion* now gives us enough evidence to lock the next product layer.

Handoff 005 turns Life Is Learned from a manually imported lesson player into a durable personal learning library:

- a remote browse/download library backed initially by GitHub Releases,
- a physical-book scanner that can locate a prepared title or submit a request,
- genuine book offloading that reclaims package/art storage while preserving learning history and collected Idea Cards,
- optional Dive Deeper content outside the ≤ 5 -minute core lesson,
- polished completion feedback that rewards finishing an idea without replacing the existing completion screen,
- an intentional app-icon/brand process,
- reader illustration sizing that matches the visual quality of the authored books,
- and cleanup of the small authoring/UI inconsistencies deferred from 004.6.

This is a platform milestone. Implement it as **sequential, independently testable epics on one branch**, not as one giant unreviewable commit.

2. Non-negotiable product rules

1. **The core learning loop remains:** Learn -> Understand -> Collect -> Revisit -> Dive Deeper.

2. The six-stage core lesson remains: **Hook -> Explanation -> Story A -> Story B -> Practical Application -> Takeaway**, then exactly two application questions.
 3. The whole core idea remains capped at **300 seconds** by the existing planning/release timing policy.
 4. **Dive Deeper does not count toward the 300-second core.** It is optional post-core content.
 5. A book package remains the **atomic distribution unit**. Never download/install one idea at a time.
 6. Stable Catalog 001 book.id values must not change.
 7. Collected Idea Cards belong to the learner and survive package updates and offload.
 8. Offload must actually reclaim heavy package/art bytes. A UI-only removal that leaves old package snapshots on disk is not acceptable.
 9. Remote discovery must not require downloading the full lesson package just to browse metadata.
 10. The first distribution transport is **GitHub Releases + a small metadata catalog**. Do not couple the package format to GitHub so transport can change later.
 11. Manual JSON import remains supported.
 12. Handoff 005 must remain backward-compatible with existing valid installed collections and learner progress.
-

3. Implementation sequence

Use these sub-epics and keep each one green before moving forward:

1. **005A - Reader/content-contract cleanup + completion feedback**
2. **005B - Library-v3 persistence, history, and real offload**
3. **005C - Remote discovery catalog + download/install manager**
4. **005D - Physical-book scanner + editorial request pipeline**
5. **005E - Dive Deeper**
6. **005F - App icon / brand gate + final integration evidence**

Do not skip ahead if an earlier persistence/data contract is still unstable.

4. 005A - Reader and authoring cleanup

4.1 Fix the illustration sizing confirmed by device testing

Current ReaderView.swift contains this behavior:

LessonIllustration(page: session.page, assets: session.assets)

.frame(maxHeight: session.lesson.usesSixStageProgress && session.page.kind != .story ?
110 : nil)

That fixed 110-point ceiling is the reason Hook, Explanation, and Practical Application art appears like a thumbnail while Story art looks correct.

Required new visual system

For canonical six-stage lessons:

Hook / Explanation / Practical Application

- Use the **same large editorial artwork container**.
- Near full width of the reading column on iPhone.
- Target approximately **16:10 / 3:2 landscape behavior**, using `aspectRatio(contentMode: .fill)` with clipping.
- Consistent corner radius and spacing across all three stages.
- Do not preserve the 110-point cap.
- On iPhone, target roughly a 180-230 point visual height depending on available width; implementation may be responsive rather than hard-coded.
- On iPad, cap the image width so it does not become an enormous poster inside a wide reading column.

Story A / Story B

- Preserve the current large cinematic treatment.
- Story remains the only stage intentionally permitted to extend below the normal screen bottom at default text size.
- Existing narration-follow auto-scroll remains Story-only.

Takeaway

- The Idea Card is the visual treatment.
- Separate artwork is **not required**.

Default-screen fit rule

At normal Dynamic Type on supported iPhones:

- Hook, Explanation, Practical Application, and Takeaway should be authored/layouted to fit without intentional scrolling.
- Story may overflow and scroll.
- Accessibility Dynamic Type and smaller available windows must remain scroll-safe; never clip content to satisfy the rule.

Acceptance screenshots

Capture at least one canonical lesson showing:

- Hook with large artwork,
- Explanation with large artwork,
- Story with current large treatment,
- Practical Application with large artwork,
- Takeaway card.

The first, second, and fifth content screens should no longer visually read as tiny thumbnails.

4.2 Support up to two images on Story screens

The production rule is now:

- Hook: exactly one primary illustration for new authored content.
- Explanation: exactly one primary illustration.
- Story A: one required illustration, **optional second illustration**.
- Story B: one required illustration, **optional second illustration**.
- Practical Application: exactly one primary illustration.
- Takeaway: separate illustration not required; the card is the visual.

Use a backward-compatible additive model. A narrow implementation is preferred over redesigning the entire package schema.

Recommended shape:

```
struct LessonPage: Codable, Identifiable, Sendable {
    // existing fields...
    var imageID: String?
    var imageDescription: String?
    var secondaryImageID: String? = nil
    var secondaryImageDescription: String? = nil
}
```

Rules:

- secondaryImageID is allowed only for .story pages in newly authored content.
- If present, it must resolve to the shared asset table and have a nonempty description.
- Render the primary Story image before the narrated body and the secondary Story image after the body, so a longer Story can visually change as the learner scrolls.
- Both images participate in lesson revision/content comparison.
- Changing secondary image bytes requires the same idea-revision bump as changing the primary image.
- Do not duplicate exact image bytes within the same idea.

Legacy packages without the new fields continue to decode unchanged.

4.3 Update the Handoff 004.6 authoring gate

The current gate requires art on all six pages. That no longer matches the product rule.

For new canonical authored content, --authoring-gate should require:

intro : 1 primary image
explanation : 1 primary image
story : 1 primary image + optional second
story : 1 primary image + optional second
application : 1 primary image
takeaway : 0 or 1 image accepted; 0 is the normal production standard

Keep:

- exact six-page semantic sequence,
- exact guide / guide / storyteller / storyteller / guide / guide role sequence,
- exactly two questions,
- <=300-second planning gate,
- image byte/dimension/type budgets,
- duplicate resolved-image protection.

For new production, emit an editorial warning if Takeaway art is supplied unnecessarily, but do **not** reject valid legacy/reviewed packages because Takeaway art exists.

Also update the authoring documentation to say clearly:

Five interior illustrations are the normal minimum per idea. Takeaway uses the collectible Idea Card. A Story page may carry a second illustration when the story length or change of scene genuinely benefits from it.

4.4 Explicitly label original fictional scenarios

Add an optional backward-compatible field:

var isOriginalFiction: Bool? = **nil**

Authoring-gate rule:

- A Story page with empty sourceIDs must explicitly set isOriginalFiction: true.
- A sourced Story may omit it or set false.
- Old stored content remains readable without the field.

This is an editorial integrity marker, not a runtime permission system.

4.5 Clean up deferred six-stage details

Small fixes that should be completed here:

- Replace stale narration-role labels such as Guide · intro & takeaway and Storyteller · lesson body with timeless UI labels such as Guide voice and Storyteller voice.
 - Canonical six-stage detection should consider the intended stage sequence and voice-role sequence so malformed content falls back safely rather than receiving canonical presentation accidentally.
 - Preserve legacy per-page progress for older lessons.
-

5. 005A - Completion accomplishment feedback

The existing completion screen stays. Do **not** replace it with a new full-screen celebration.

5.1 Completion effect: "Insight Bloom"

When an idea transitions from not-complete to practice-complete for the first time on the active revision:

1. Play a distinct success haptic.
2. Play a short, warm completion chime when sound effects are enabled.
3. Layer a brief visual celebration over the existing completion view.

Preferred visual direction:

- soft radial glow around the completion/checkmark area,
- one expanding ring,
- a small number of teal/gold spark or card-like particles,
- approximately 0.8-1.3 seconds,
- no screen-filling confetti,
- no obstruction of Continue / View Card controls,
- no repeated looping.

This should feel like **an idea locking into the learner's collection**, not a casino reward.

5.2 Trigger rules

Celebrate:

- first completion of a new idea revision.

Do not fire the full celebration:

- every time the learner opens an already-completed review,
- when merely viewing the Idea Card,

- after a failed or incomplete practice attempt.

5.3 Accessibility and settings

Add completion feedback preferences to existing settings:

- Completion haptics (default on where supported)
- Completion sound (default on)

Behavior:

- audio must respect the device's normal silent/mixing expectations; use an ambient/non-intrusive audio-session strategy,
- Reduce Motion replaces particle travel with a restrained glow / ring state,
- if Core Haptics is unavailable, fall back to a standard success notification haptic,
- celebration failure must never block lesson completion or card earning.

Add deterministic tests around "fires once on transition" rather than trying to assert subjective haptic quality.

6. 005B - Library-v3 persistence and real offload

This is the highest-risk architecture work in Handoff 005.

6.1 Why v2 cannot simply gain an Offload button

The current Library-v2 snapshot model stores full CollectionCatalog objects in historical collections.json snapshots. IdeaCardStorage.archivedSource(for:) searches those historical packages to reconstruct a full lesson.

Therefore removing a book only from the active catalog could leave old prose and image bytes in historical snapshots and reclaim little or no storage.

Do not ship Offload on top of that behavior.

6.2 Storage goals

Separate durable learner state from removable distribution payload.

Durable learner state:

- LessonProgress for every retained revision,
- IdeaCardRecord and favorite state,
- lightweight book history metadata,
- installed/source metadata,
- stable IDs and revision references.

Removable payload:

- lesson prose package,
- package source definitions,
- full cover bytes,
- lesson artwork bytes,
- obsolete downloaded package revisions.

6.3 Recommended Library-v3 shape

A concrete implementation may vary, but preserve this separation:

```
Library-v3/  
  CURRENT.json  
  StateSnapshots/  
    <uuid>/  
      library-state.json  
      progress.json  
      cards.json  
      history.json  
  Packages/  
    <bookID>/  
      <collectionRevision>-<sha256>.json  
  Downloads/  
    staging/
```

library-state.json should contain **references/metadata**, not embedded LessonPackage payloads.

Suggested installed record:

```
struct InstalledBookRecord: Codable, Sendable {  
  var bookID: String  
  var collectionRevision: Int  
  var packageFile: String  
  var packageSHA256: String  
  var packageBytes: Int  
  var source: BookSourceRecord  
}
```

Suggested durable history:

```
struct BookHistoryRecord: Codable, Identifiable, Sendable {  
  var bookID: String  
  var title: String  
  var author: String
```



```

var firstSeenAt: Date?
var lastOpenedAt: Date?
var firstCompletedAt: Date?
var lastKnownCollectionRevision: Int
var lastKnownIdeaCount: Int
var lastKnownPracticedCount: Int
var source: BookSourceRecord
var id: String { bookID }
}

```

BookSourceRecord must distinguish at least:

- manual import,
- remote catalog/download.

Remote records should preserve enough catalog identity to offer a future re-download. Manual records should explain that re-import is required.

6.4 Migration from Library-v2

Migration must be crash-safe:

1. Read the current v2 library and learner state.
2. Complete any Idea Card migration that still depends on historical package snapshots.
3. Write v3 package files for **currently installed packages only**.
4. Write v3 progress/cards/history/state snapshots.
5. Validate the v3 round-trip and identity counts.
6. Atomically publish v3 CURRENT.json.
7. Only after a verified successful v3 launch, remove the obsolete Library-v2 package snapshots so storage is genuinely reclaimed.

If migration fails before publication, v2 remains untouched and usable.

Do not retain a permanent full v2 backup after successful migration; that would defeat offload storage reclamation.

6.5 History behavior

A lightweight History view must retain books the learner started or completed even when the package is offloaded.

At minimum show:

- title,
- author,
- practiced ideas / last-known idea count,
- completed date when known,

- installed/offloaded state.

If a previously completed book later receives an updated release with new or revised ideas, preserve firstCompletedAt and show that the current version has updates rather than erasing the original completion event.

6.6 Offload behavior

Add an Offload Book action from Book Detail / book menu.

Before confirmation, show:

- approximate storage to be reclaimed,
- that Idea Cards, favorites, progress, and History remain,
- remote books can be downloaded again,
- manual books must be re-imported from a file.

On confirmation:

- remove the active installed-package reference,
- delete unreferenced package payload for that book,
- garbage-collect obsolete package files not referenced by any installed state,
- keep progress/cards/history,
- return to a sensible library/history destination.

The operation must be atomic from the learner's point of view. If deletion fails, keep the book installed and report the error.

6.7 Idea Card behavior after offload

Cards remain fully browsable from their snapshots.

Remove dependence on archivedSource(for:) for normal card browsing.

When the learner asks to review the **full lesson** behind an offloaded card:

- remote source -> offer Download current book;
- manual source -> offer Re-import book;
- if the current downloadable revision is newer than the earned card revision, say the current lesson may differ; do not rewrite the earned card snapshot.

IdeaCardCollection.ordered must continue to produce useful book ordering even when the active LearningBook is absent; use History metadata rather than treating all offloaded cards as unknown-tail items.

6.8 Required storage evidence

Provide a deterministic test fixture with a package containing enough artwork to measure reclamation.

Prove:

- installed package consumes expected payload bytes,
- offload removes those payload bytes,
- cards/progress/history remain byte-for-byte/semantically intact,
- reinstall reconnects to existing progress without duplicate cards,
- failed offload leaves installed content intact,
- migration v2 -> v3 preserves all current learning state.

"Book disappeared from UI" is not evidence of successful offload.

7. 005C - Remote discovery catalog and GitHub Releases

7.1 Keep discovery separate from installed CollectionCatalog

The existing CollectionCatalog represents installed packages/revisions and learner-facing local state. **Do not repurpose it.**

Use a distinct type such as:

```
struct DiscoveryCatalog: Codable, Sendable { ... }  
struct DiscoveryBook: Codable, Identifiable, Sendable { ... }
```

The remote catalog is editorial/discovery metadata only.

7.2 Remote catalog schema

Add a versioned document such as:

Catalog/Remote-Catalog-001.json

Suggested shape:

```
{  
  "schemaVersion": 1,  
  "catalogID": "catalog-001",  
  "catalogRevision": 1,  
  "books": [  
    {  
      "id": "thinking-fast-and-slow",  
      "title": "Thinking, Fast and Slow",  
      "authors": ["Daniel Kahneman"],  
      "primaryShelfID": "psychology-human-behavior",  
      "secondaryShelfIDs": [],  
      "tags": ["psychology", "decision-making"],  
    },  
  ],  
}
```

```

    "isbn13": ["..."],
    "availability": "available",
    "thumbnail": {
      "url": "https://.../cover.jpg",
      "sha256": "...",
      "bytes": 123456
    },
    "package": {
      "collectionRevision": 1,
      "url": "https://github.com/.../releases/download/.../book.json",
      "sha256": "...",
      "bytes": 7421130
    }
  }
]
}

```

Allowed availability states should be documented, for example:

- available
- planned
- unavailable

Do not encode request counts in the public static catalog unless the editorial backend intentionally publishes them.

Catalog principles

- browsing downloads metadata only,
- cover/thumbnail is a small separate asset,
- package URL is optional unless available,
- checksums and byte sizes are mandatory for downloadable packages,
- book.id must exactly match Catalog 001,
- package format remains formatVersion 2,
- transport URLs are metadata, not embedded package assumptions.

7.3 First transport

Use public GitHub Releases for package files.

Do not publish releases automatically from Codex in this handoff. Implement and test the transport/tooling; reviewer controls public publication.

Provide a helper script that can generate/verify remote catalog package metadata from a local final JSON:

- SHA-256,

- byte size,
- collection revision,
- stable book ID,
- optional thumbnail metadata.

The helper must never silently rewrite stable IDs.

7.4 Discovery UI

Keep the current home/library clean.

Preferred add-book flow:

The existing + opens an **Add Books** surface with:

- Browse Library
- Scan a Book
- Import File

Browse Library opens a searchable/filterable Discovery Catalog:

- title/author search,
- shelf filters,
- cover thumbnail or editorial placeholder,
- In Library, Download, Update, Coming Soon, or Request state.

Browsing must not download the package.

Manual Import File keeps the existing file importer behavior.

7.5 Catalog caching/offline behavior

- Cache the last successfully decoded discovery catalog.
- Use the cached catalog when offline.
- A bad network response must not destroy the cached catalog.
- Expose a subtle stale/offline indicator when appropriate.
- Catalog refresh should be cancellable and must not block the installed library from opening.

The catalog endpoint should be configurable rather than hard-coded into multiple views.

8. 005C - Download and install manager

Implement a dedicated download boundary, preferably an actor/service rather than networking inside views.

Required behavior:

- download to a staging file,
- visible progress,
- cancel,
- resume when URLSession supplies usable resume data,
- validate expected byte size when supplied,
- validate SHA-256 before decode/import,
- validate package using existing package rules,
- perform atomic install through Library-v3,
- remove staging files after success/failure as appropriate.

A checksum mismatch must never reach the installed library.

Update behavior

For a remote book already installed:

- compare catalog collection revision with installed revision,
- expose Update only for a newer prepared release,
- preserve Handoff 002/003 revision semantics,
- if the update declares removed idea IDs, preserve the existing acknowledgement flow before commit.

Source identity

After successful remote install, save the discovery source metadata into BookSourceRecord so Offload can later offer a redownload.

9. 005D - Physical-book scanner

9.1 Product goal

The scanner is a discovery shortcut, not an AI lesson generator.

Flow:

Camera -> identify physical book -> match Discovery Catalog

-> available: show prepared book + Download

-> catalog match but unavailable: Request this book

-> unknown: review recognized title/author/ISBN -> Request this book

Never generate an instant lesson package from the scan.

9.2 Recognition strategy

Use native Apple frameworks. Prefer reliable structured identifiers before fuzzy cover text:

1. Detect an ISBN/EAN-13 barcode when visible.
2. Also recognize title/author text from the cover using live text recognition.
3. Match locally against the already-downloaded Discovery Catalog.
4. Rank exact ISBN match above normalized title/author fuzzy matching.
5. Show the learner the proposed match before taking action.

A custom camera UI may wrap VisionKit/Vision functionality, but it must visually belong to Life Is Learned rather than looking like a generic system scanner.

Scanner privacy

- Perform recognition on device where practical.
 - Do not upload camera frames to the editorial request backend.
 - Only submit reviewed book metadata when the learner explicitly taps Request.
-

10. 005D - Editorial book-request pipeline

A public app cannot safely embed a GitHub write token. Do not put repository credentials in the app.

Provide an implementation-neutral client protocol plus a minimal reference backend under Backend/RequestAPI/.

A small serverless backend is appropriate. The reference implementation may use Cloudflare Worker + D1 (or an equivalently small serverless store), but app code must depend only on the HTTP contract.

10.1 Request contract

Suggested endpoint:

POST /v1/book-requests

Content-Type: application/json

Body:

```
{
  "catalogBookID": "optional-known-id",
  "title": "Recognized or selected title",
  "author": "optional author",
  "isbn13": "optional isbn",
  "source": "scanner"
}
```

Response:

```
{
  "status": "accepted",
  "requestKey": "normalized-editorial-key",
  "requestCount": 14
}
```

Server behavior:

- normalize/dedupe the same requested title/ISBN,
- increment request count,
- store first/last request time,
- expose an editorial queue through a protected/admin path or documented database query,
- rate-limit obvious spam,
- do not require personal account data for a simple request.

The app should show a calm success state such as Requested · we'll use requests to prioritize the library.

Do not deploy production infrastructure automatically from this branch. Include source, config template, tests, and deployment instructions. Production URL/secret provisioning is a reviewer-controlled release step.

11. 005E - Dive Deeper

Dive Deeper is optional content unlocked **after the core idea has been practiced/collected**.

It must never make the core lesson longer.

11.1 Data model

Add an optional backward-compatible lesson field, for example:

```
struct DiveDeeperSection: Codable, Identifiable, Sendable {
  var id: String
  var title: String
  var text: String
  var sourceIDs: [String]
}

struct DiveDeeperContent: Codable, Sendable {
  var title: String
  var summary: String?
  var sections: [DiveDeeperSection]
```



```
}
```

```
struct Lesson {  
  // existing fields...  
  var diveDeeper: DiveDeeperContent? = nil  
}
```

Rules:

- optional; legacy and current books decode without it,
- every non-fiction factual section uses valid source IDs,
- not included in the ≤ 300 -second core timing gate,
- included in deterministic lesson-content revision comparison because changing reviewed deeper content changes the idea package,
- no requirement to create Dive Deeper for every idea.

11.2 UI entry points

When available and the idea is earned:

- completion screen may show Dive deeper,
- Idea Card detail may show Dive deeper.

Do not show it as a seventh core progress stage.

11.3 Dive Deeper view

Use the same editorial visual language but a clearly different mode:

- scrollable sections,
- examples,
- edge cases,
- connections to other ideas,
- practical application.

Guide-voice read-aloud may reuse the existing speech player, but Dive Deeper is learner-controlled and has no automatic five-minute deadline.

If an offloaded card has Dive Deeper only inside the removed package, offer redownload/re-import rather than pretending the deeper content remains local.

12. 005F - App icon and brand gate (Option B)

The app must leave Handoff 005 with a real, intentional icon. Do not accept the default/blank icon.

This is a **design gate**, not a request for Codex to invent a random final graphic.

12.1 Brand attributes

The icon should communicate:

- collected insight,
- books/ideas without looking like a generic ebook reader,
- intelligent and editorial,
- warm rather than clinical,
- calm confidence rather than gamified noise.

Current product palette provides the starting family:

- deep ink / deep teal,
- warm ivory,
- restrained warm-gold accent.

Avoid:

- graduation caps,
- literal brain clip-art,
- obvious AI sparkle logos,
- tiny text,
- busy multi-object illustrations,
- visual similarity to Kindle/Apple Books/Deepstash.

12.2 Required design process

Before final asset insertion, produce three concise icon directions for reviewer choice.
Recommended conceptual territory:

Direction A - Idea Fold

A page/card fold that creates a subtle L or path shape; editorial and architectural.

Direction B - Collected Insight

A small stack of rounded idea cards/book layers with one restrained insight node; emphasizes collection and learning.

Direction C - Learning Orbit

A minimal page/idea form with one orbit/path element; connects reading with ideas that remain and compound.

Reviewer selects/refines one direction before the final 1024x1024 source is produced.

12.3 Production icon requirements

Final AppIcon assets must include appropriate modern iOS variants:

- standard/Any,

- Dark,
- Tinted where supported by the deployment target/toolchain.

Requirements:

- legible at small home-screen size,
- no manually pre-rounded outer corners,
- no text that must be read,
- central mark stays inside safe visual margins,
- icon works against both light and dark system contexts,
- include source/brief documentation in Brand/ but do not ship unnecessary design working files in the app bundle.

Merge is blocked until the reviewer-approved icon is actually present in the asset catalog and visible in a device/simulator evidence screenshot.

13. Remote + local UI architecture

Do not turn the app into a tab-heavy storefront.

Preferred navigation:

Home / Your Library remains the primary landing experience.

Toolbar/add flow:

- + Add Books
 - > Browse Library
 - > Scan a Book
 - > Import File

Keep Ideas accessible from the existing Ideas control.

Add History in a way that does not dominate the primary learning surface. Good options include:

- a compact History entry near Your Library,
- a book-management menu/destination,
- or a secondary section below active library content.

The reviewer should not have to wade through a store screen to resume learning.

14. Source and availability states

The UI must be able to distinguish:

remote + not installed + available	-> Download
remote + installed + current	-> In Library
remote + installed + newer release	-> Update
remote + offloaded + available	-> Download Again
manual + installed	-> In Library
manual + offloaded	-> Re-import Required
catalog known + unavailable	-> Request
unknown scan	-> Review metadata -> Request

Do not infer availability solely from the presence of a book in Catalog 001. Catalog 001 is editorial planning; Discovery Catalog says what is actually downloadable.

15. Security, privacy, and failure behavior

- Never put GitHub write tokens or backend admin secrets in the iOS app.
 - Validate downloaded SHA-256 before decoding/importing.
 - Scanner images stay on device; request API receives reviewed metadata only.
 - A failed catalog refresh never deletes the last good cache.
 - A failed package download never mutates the installed library.
 - A failed offload never discards learner state.
 - A failed v2 -> v3 migration never destroys the v2 source library.
 - Remote URLs must use HTTPS.
 - Request endpoint must have sane request size/rate limits.
-

16. Tests required

16.1 Reader / authoring

- canonical Hook/Explanation/Application use the new large editorial frame,
- Story retains large scrolling treatment,
- Takeaway does not require art,
- one and two-image Story layouts render correctly,
- secondary Story image is included in deterministic revision comparison,
- authoring gate accepts five-primary-image standard,
- missing Hook/Explanation/Story/Application image fails,
- duplicate bytes in one idea fail,
- fictional Story with empty sources requires explicit fiction marker,

- legacy packages still decode.

16.2 Completion feedback

- celebration fires once on incomplete -> complete transition,
- revisit does not replay full first-completion celebration,
- updated revision may celebrate when newly completed,
- disabled sound/haptics are respected,
- Reduce Motion selects restrained visual path,
- feedback-service error cannot block completion/card earning.

16.3 Library-v3 / offload

- v2 migration preserves package identities, progress, cards, favorites,
- migration interruption leaves v2 usable,
- successful cleanup removes old v2 payload after v3 verification,
- offload deletes package bytes,
- offload preserves progress/cards/history,
- manual-offloaded and remote-offloaded redownload/reimport messaging differs correctly,
- reinstallation reconnects state without duplicate cards,
- History retains completion date,
- card ordering remains stable with offloaded books,
- garbage collector never deletes referenced installed package files.

16.4 Discovery/download

- catalog decode + schema/version rejection,
- last-good catalog cache survives malformed refresh,
- search/shelf filters,
- available/planned/in-library/update states,
- progress and cancel,
- checksum mismatch fails before install,
- byte-size mismatch fails safely,
- successful download installs atomically,
- update respects removal acknowledgement.

Use URLProtocol/test fixtures rather than real public GitHub networking in deterministic unit tests.

16.5 Scanner/request

- exact ISBN catalog match outranks fuzzy title match,
- normalized title/author matching,
- ambiguous result asks learner to choose,
- unknown result can be edited before request,
- no request is sent until explicit learner action,

- request success/duplicate/rate-limit/network-error states,
- camera permission denied has a useful fallback to text search/manual entry.

16.6 Dive Deeper

- old package without Dive Deeper decodes,
- locked before idea completion,
- visible after idea completion,
- source validation,
- not included in core ≤ 300 -second timing calculation,
- content changes require idea revision bump,
- offloaded deeper content requests reinstall/reimport.

16.7 Brand

- AppIcon asset set is present,
 - build has no missing-icon warning,
 - simulator/device evidence shows the icon,
 - dark/tinted variants compile where supported.
-

17. Evidence package

Create:

Evidence/Handoff005/
 README.md
 005A-reader/
 005A-completion/
 005B-storage-offload/
 005C-discovery-download/
 005D-scanner-request/
 005E-dive-deeper/
 005F-brand/
 test-results.json

Evidence must include:

Reader

- before/after device screenshots proving non-story art is no longer tiny,
- one two-image Story fixture,
- Takeaway without separate art.

Completion

- screenshot or short sequence showing Insight Bloom without replacing completion UI,
- Reduce Motion state.

Storage/offload

- before/after on-disk byte counts for an art-heavy fixture,
- state dump proving cards/progress/history remain,
- reinstall state proof.

Discovery

- browse catalog,
- download progress,
- downloaded/in-library state,
- update state,
- offline cached catalog.

Scanner

- ISBN match,
- title-text match/fallback,
- unavailable/request flow.

Dive Deeper

- completion entry point,
- Idea Card entry point,
- representative section view.

Brand

- final selected AppIcon on home screen / app switcher evidence where practical.

Include exact commit SHAs and test commands.

18. Commit discipline

Recommended commits:

1. H005A reader contract and completion feedback
2. H005B migrate library storage and add offload history
3. H005C add discovery catalog and package downloads
4. H005D add scanner and request client/backend template
5. H005E add Dive Deeper
6. H005F integrate approved brand icon and evidence

Do not collapse the full milestone into one giant commit.

After every epic:

- run targeted unit tests,

- run relevant UI tests,
- commit only when green,
- continue on the same Handoff 005 branch.

At the end, run the full regression suite and stop for reviewer approval.

19. Public deployment gate

Implementation can be approved before public endpoints are deployed, but **production publishing** requires the following reviewer-controlled steps:

1. Publish approved book JSON/thumbnail assets to GitHub Releases.
2. Publish/configure the Discovery Catalog URL.
3. Deploy/configure the request API endpoint and secrets.
4. Populate app build configuration with production URLs.
5. Run one real-network smoke test for catalog -> download -> checksum -> install -> offload -> redownload.
6. Confirm the selected AppIcon asset is the production one.

Codex must not create public releases, deploy external services, or expose credentials without explicit reviewer/user authorization.

20. Out of scope for Handoff 005

Do not add unless required for a blocker:

- user accounts,
- cross-device cloud sync,
- subscriptions/paywall,
- ratings/reviews/community comments,
- instant AI-generated book lessons,
- DRM,
- arbitrary web scraping of books,
- background auto-download of books,
- remote deletion of learner data,
- mass publication of the 50-book Catalog 001 library.

Mass content production resumes **after Handoff 005 is accepted**.

21. Reviewer acceptance summary

Handoff 005 is accepted when all of the following are true:

- Hook/Explanation/Practical Application artwork is large and visually consistent on device; Story remains the larger scrolling treatment.
- Story pages can optionally carry a second illustration.
- Takeaway/Idea Card no longer requires redundant illustration art in the authoring gate.
- original-fiction Story provenance can be explicitly represented.
- completion produces a tasteful one-time Insight Bloom, haptic, and optional sound without replacing the completion screen.
- Library-v3 separates learner state from package payload.
- Offload proves real byte reclamation while preserving cards, favorites, progress, and History.
- remote catalog browsing works without package download.
- GitHub Release downloads show progress, validate checksum/size, and install atomically.
- scanner can match by ISBN/text and never generates instant lessons.
- unavailable/unknown books can enter an editorial request queue through a credential-safe backend.
- Dive Deeper exists outside the ≤ 5 -minute core and is accessible after an idea is earned.
- manual import still works.
- existing installed books/progress survive migration.
- a reviewer-approved production AppIcon is actually integrated.
- full regression/evidence passes.

After 005 approval, the project is ready to resume the library-production pipeline in quality-controlled two-book batches.